

GenAi Analysis

Name of AI #1:

Copilot

Accessed:

On my computer by searching for the website.

Prompt:

I need a C program that manages a CEO's email inbox using a MaxHeap priority queue. The heap must be built completely from scratch using a list/array—no built-in heap libraries. Each email has a sender category, subject, and date (MM-DD-YYYY). Priority is based first on the sender category (Boss highest, then Subordinate, Peer, ImportantPerson, and OtherPerson) and then by date, where newer emails should be treated as higher priority. The program should read commands from input: "EMAIL <sender>,<subject>,<date>" to add an email, "NEXT" to show the highest-priority email without removing it, "READ" to remove the top email, and "COUNT" to show how many emails are currently in the queue. It should handle edge cases like NEXT or READ when the queue is empty, multiple NEXTs in a row, and multiple READs in a row. The MaxHeap must include insert, extractMax, heapify-up, and heapify-down functions. The program should keep processing commands until EOF and then exit normally. Generate one complete, compilable C file that does all of this.

Generated Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_STR 200

// -----
// Email Structure
// -----
typedef struct {
    char sender[MAX_STR];
    char subject[MAX_STR];
    char date[MAX_STR]; // MM-DD-YYYY
    int priority;        // Computed priority
} Email;

// -----
// MaxHeap Structure
// -----
typedef struct {
    Email *data;
```

```

    int size;
    int capacity;
} MaxHeap;

// -----
// Priority Mapping
// -----
int senderPriority(const char *sender) {
    if (strcmp(sender, "Boss") == 0) return 50000;
    if (strcmp(sender, "Subordinate") == 0) return 40000;
    if (strcmp(sender, "Peer") == 0) return 30000;
    if (strcmp(sender, "ImportantPerson") == 0) return 20000;
    return 10000; // OtherPerson
}

// Convert MM-DD-YYYY → integer YYYYMMDD
int dateToInt(const char *date) {
    int mm, dd, yyyy;
    sscanf(date, "%d-%d-%d", &mm, &dd, &yyyy);
    return yyyy * 10000 + mm * 100 + dd;
}

// Compute final priority
int computePriority(const char *sender, const char *date) {
    return senderPriority(sender) + dateToInt(date);
}

// -----
// Heap Utility Functions
// -----
MaxHeap* createHeap(int capacity) {
    MaxHeap *heap = malloc(sizeof(MaxHeap));
    heap->data = malloc(sizeof(Email) * capacity);
    heap->size = 0;
    heap->capacity = capacity;
    return heap;
}

void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

```

```

void heapifyUp(MaxHeap *heap, int index) {
    int parent = (index - 1) / 2;
    while (index > 0 && heap->data[index].priority > heap->data[parent].priority) {
        swap(&heap->data[index], &heap->data[parent]);
        index = parent;
        parent = (index - 1) / 2;
    }
}

void heapifyDown(MaxHeap *heap, int index) {
    int left, right, largest;
    while (1) {
        left = index * 2 + 1;
        right = index * 2 + 2;
        largest = index;

        if (left < heap->size && heap->data[left].priority > heap->data[largest].priority)
            largest = left;
        if (right < heap->size && heap->data[right].priority > heap->data[largest].priority)
            largest = right;

        if (largest == index) break;

        swap(&heap->data[index], &heap->data[largest]);
        index = largest;
    }
}

void insertEmail(MaxHeap *heap, Email email) {
    if (heap->size == heap->capacity) {
        heap->capacity *= 2;
        heap->data = realloc(heap->data, sizeof(Email) * heap->capacity);
    }
    heap->data[heap->size] = email;
    heapifyUp(heap, heap->size);
    heap->size++;
}

Email extractMax(MaxHeap *heap) {
    Email empty = {"", "", "", -1};
    if (heap->size == 0) return empty;

    Email maxEmail = heap->data[0];
    heap->data[0] = heap->data[heap->size - 1];

```

```

    heap->size--;
    heapifyDown(heap, 0);

    return maxEmail;
}

// -----
// Print Email
// -----
void printEmail(const Email *e) {
    printf("Sender: %s\n", e->sender);
    printf("Subject: %s\n", e->subject);
    printf("Date: %s\n", e->date);
}

// -----
// Main Program
// -----
int main() {
    MaxHeap *queue = createHeap(10);
    Email currentNext = {"", "", "", -1};

    char line[MAX_STR];

    while (fgets(line, sizeof(line), stdin)) {
        if (strncmp(line, "EMAIL", 5) == 0) {
            char sender[MAX_STR], subject[MAX_STR], date[MAX_STR];

            // Parse EMAIL command
            char *ptr = strchr(line, ' ');
            ptr++;

            char *token = strtok(ptr, ",");
            strcpy(sender, token);

            token = strtok(NULL, ",");
            strcpy(subject, token);

            token = strtok(NULL, ",");
            strcpy(date, token);
            date[strcspn(date, "\n")] = 0;

            Email e;
            strcpy(e.sender, sender);

```

```

        strcpy(e.subject, subject);
        strcpy(e.date, date);
        e.priority = computePriority(sender, date);

        insertEmail(queue, e);
    }
    else if (strncmp(line, "NEXT", 4) == 0) {
        if (queue->size == 0) {
            printf("No emails.\n");
        } else {
            currentNext = queue->data[0];
            printEmail(&currentNext);
        }
    }
    else if (strncmp(line, "READ", 4) == 0) {
        if (queue->size == 0) {
            printf("No emails to read.\n");
        } else {
            extractMax(queue);
        }
    }
    else if (strncmp(line, "COUNT", 5) == 0) {
        printf("%d\n", queue->size);
    }
}

free(queue->data);
free(queue);
return 0;
}

```

Analysis:

The code is correct and successfully uses a MaxHeap to manage emails based on their priority. The execution time is efficient because inserting and removing an email takes $O(\log n)$, while NEXT and COUNT take $O(1)$ time. The space complexity is $O(n)$ because the program stores the emails in the heap. The code is also maintainable because it is organized into different functions that each handle a specific part of the program.

Improvements:

One improvement would be adding checks for incorrect EMAIL inputs so the program does not have problems with missing information. The malloc and realloc functions should also be checked to make sure the memory was allocated correctly. Another improvement would be removing extra spaces from the sender, subject, and date. These changes would make the

program more reliable while keeping the code simple. Also, the output of this code doesn't quite match the example output in the assignment 2 details. I would add proper output formatting.

Selection choice:

I chose to improve Copilot because of the lack of comments and it would be easier to improve upon. This is because the code is already organized into different functions, making it easier to understand what each part of the program does and add the necessary comments.

Name of AI #2:

Gemini

Accessed:

On my computer by searching for the website.

Prompt:

I need a C program that manages a CEO's email inbox using a MaxHeap priority queue. The heap must be built completely from scratch using a list/array—no built-in heap libraries. Each email has a sender category, subject, and date (MM-DD-YYYY). Priority is based first on the sender category (Boss highest, then Subordinate, Peer, ImportantPerson, and OtherPerson) and then by date, where newer emails should be treated as higher priority. The program should read commands from input: "EMAIL <sender>,<subject>,<date>" to add an email, "NEXT" to show the highest-priority email without removing it, "READ" to remove the top email, and "COUNT" to show how many emails are currently in the queue. It should handle edge cases like NEXT or READ when the queue is empty, multiple NEXTs in a row, and multiple READs in a row. The MaxHeap must include insert, extractMax, heapify-up, and heapify-down functions. The program should keep processing commands until EOF and then exit normally. Generate one complete, compilable C file that does all of this.

Generated Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

#define MAX_LINE 256

// Struct to store Email details
typedef struct {
    char category[32];
    char subject[128];
```

```

    char date[11]; // MM-DD-YYYY\0
} Email;

// Struct for the MaxHeap Priority Queue
typedef struct {
    Email *data;
    int size;
    int capacity;
} MaxHeap;

// Category priority hierarchy (Higher integer = Higher priority)
int get_category_priority(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0;
}

// Convert MM-DD-YYYY to YYYYMMDD integer for chronological comparison
int parse_date(const char *date) {
    int month, day, year;
    sscanf(date, "%d-%d-%d", &month, &day, &year);
    return year * 10000 + month * 100 + day;
}

// Priority Comparison: Returns >0 if e1 has higher priority than e2
int compare_emails(Email e1, Email e2) {
    int p1 = get_category_priority(e1.category);
    int p2 = get_category_priority(e2.category);

    if (p1 != p2) {
        return p1 - p2; // Higher category priority wins
    }

    // Tiebreaker: Newest email (larger YYYYMMDD date value) comes first
    return parse_date(e1.date) - parse_date(e2.date);
}

// Initialize MaxHeap
MaxHeap* create_heap(int initial_capacity) {
    MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->size = 0;

```

```

    heap->capacity = initial_capacity;
    heap->data = (Email*)malloc(sizeof(Email) * initial_capacity);
    return heap;
}

```

```

void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

```

// Heapify Up (Insertion)

```

void heapify_up(MaxHeap *heap, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (compare_emails(heap->data[index], heap->data[parent]) > 0) {
            swap(&heap->data[index], &heap->data[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

```

// Heapify Down (Deletion/Extraction)

```

void heapify_down(MaxHeap *heap, int index) {
    while (2 * index + 1 < heap->size) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < heap->size && compare_emails(heap->data[left], heap->data[largest]) > 0) {
            largest = left;
        }
        if (right < heap->size && compare_emails(heap->data[right], heap->data[largest]) > 0) {
            largest = right;
        }

        if (largest != index) {
            swap(&heap->data[index], &heap->data[largest]);
            index = largest;
        } else {
            break;
        }
    }
}

```

```
}  
}
```

```
// Insert Email into Priority Queue
```

```
void heap_insert(MaxHeap *heap, Email email) {  
    if (heap->size == heap->capacity) {  
        heap->capacity *= 2;  
        heap->data = (Email*)realloc(heap->data, sizeof(Email) * heap->capacity);  
    }  
    heap->data[heap->size] = email;  
    heapify_up(heap, heap->size);  
    heap->size++;  
}
```

```
// Remove Highest Priority Email (READ)
```

```
void heap_delete_max(MaxHeap *heap) {  
    if (heap->size == 0) return;  
    heap->data[0] = heap->data[heap->size - 1];  
    heap->size--;  
    if (heap->size > 0) {  
        heapify_down(heap, 0);  
    }  
}
```

```
// Peek Highest Priority Email (NEXT)
```

```
bool heap_peek_max(MaxHeap *heap, Email *result) {  
    if (heap->size == 0) return false;  
    *result = heap->data[0];  
    return true;  
}
```

```
void free_heap(MaxHeap *heap) {
```

```
    free(heap->data);  
    free(heap);  
}
```

```
int main(int argc, char *argv[]) {
```

```
    FILE *file = stdin;  
    if (argc > 1) {  
        file = fopen(argv[1], "r");  
        if (!file) {  
            perror("Failed to open input file");  
            return 1;  
        }  
    }
```

```

}

MaxHeap *queue = create_heap(10);
char line[MAX_LINE];

while (fgets(line, sizeof(line), file)) {
    // Strip trailing newline character
    line[strcspn(line, "\r\n")] = 0;

    if (strncmp(line, "EMAIL ", 6) == 0) {
        Email new_email;
        char *token_str = line + 6;

        // Parse comma-separated fields: category, subject, date
        char *cat = strtok(token_str, ",");
        char *sub = strtok(NULL, ",");
        char *dt = strtok(NULL, ",");

        if (cat && sub && dt) {
            // Trim leading spaces
            while (*cat == ' ') cat++;
            while (*sub == ' ') sub++;
            while (*dt == ' ') dt++;

            strncpy(new_email.category, cat, sizeof(new_email.category));
            strncpy(new_email.subject, sub, sizeof(new_email.subject));
            strncpy(new_email.date, dt, sizeof(new_email.date));

            heap_insert(queue, new_email);
        }
    } else if (strcmp(line, "NEXT") == 0) {
        Email next_email;
        if (heap_peek_max(queue, &next_email)) {
            printf("Sender:%s\n", next_email.category);
            printf("Subject:%s\n", next_email.subject);
            printf("Date:%s\n", next_email.date);
        }
    } else if (strcmp(line, "READ") == 0) {
        heap_delete_max(queue);
    } else if (strcmp(line, "COUNT") == 0) {
        printf("%d\n", queue->size);
    }
}

```

```
if (file != stdin) {  
    fclose(file);  
}  
free_heap(queue);  
return 0;  
}
```

Analysis:

The code is correct and successfully implements a MaxHeap priority queue for managing emails. The execution time is efficient because inserting and removing an email takes $O(\log n)$, while checking the next email and counting emails take $O(1)$ time. The space complexity is $O(n)$ because the heap stores all of the emails in memory. The code is also maintainable because it is separated into different functions and includes comments explaining the main sections of the program.

Improvements:

The parsing could be more reliable by removing spaces at the beginning and end of categories and subjects. This would help prevent extra spaces from causing problems when comparing them or displaying the output. Adding checks for incorrect EMAIL lines and dates would also help the program handle unexpected input better. Adding a space after each colon in the NEXT output would make it easier to read. Also, like I said for the last Ai the output doesn't match perfectly with the example given in the assignment details.